



Faculty of Science



Parallel Basic Blocks and Flattening Nested Parallelism

Cosmin E. Oancea

`cosmin.oancea@diku.dk`

Department of Computer Science (DIKU)
University of Copenhagen

September 2026 PMPH Lecture Slides



1 Implementation of Flat Bulk Operators

- Amdahl's Law
- Work-Depth Asymptotic
- Implementation of Reduce
- Implementation of Scan
- Implementation of Segmented Scan
- Other Second-Order Parallel Operators

2 Nested Data-Parallel Applications

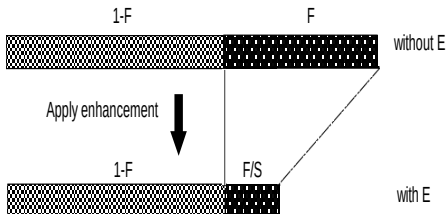
- Sieve: Prime-Numbers Computation
- Nested Parallel Quicksort

3 Flattening Nested Parallelism

- Flattening Recipe (by Map Distribution)
- Re-Writing Rules For Flattening
- Flattening Prime-Number (Sieve) Computation
- Flattening Quicksort



Amdahl's Law



Enhancement accelerates a fraction F of the task by a factor S :

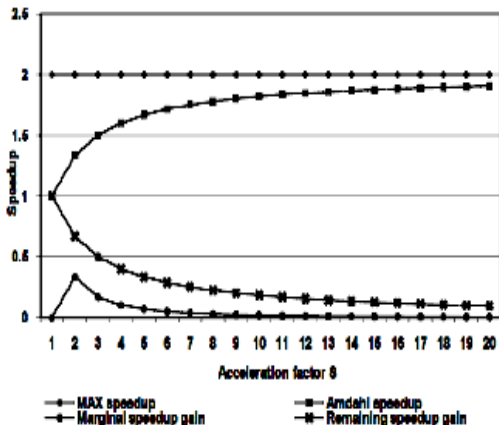
$$T_{\text{exe}}(\text{with}E) = T_{\text{exe}}(\text{without}E) \times \left[(1 - F) + \frac{F}{S} \right]$$

$$\text{Speedup}(E) = \frac{T_{\text{exe}}(\text{without}E)}{T_{\text{exe}}(\text{with}E)} = \frac{1}{(1-F) + \frac{F}{S}}$$



Amdahl's Law

- 1 Improvement is limited by the $1 - F$ part of the execution that cannot be optimized: $Speedup(E) < \frac{1}{1-F}$
- 2 Optimize the common case & execute the rare case in software.
- 3 Low of diminishing returns

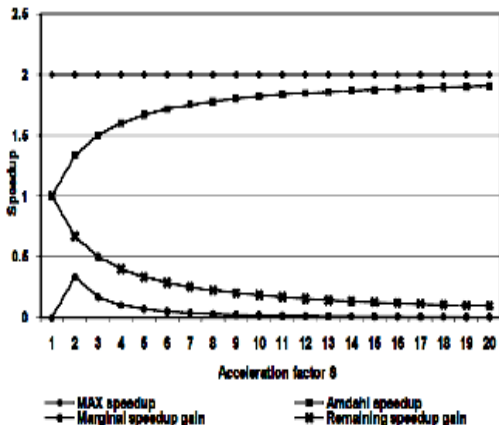


$F=0.5$



Amdahl's Law

- 1 Improvement is limited by the $1 - F$ part of the execution that cannot be optimized: $Speedup(E) < \frac{1}{1-F}$
- 2 Optimize the common case & execute the rare case in software.
- 3 Low of diminishing returns



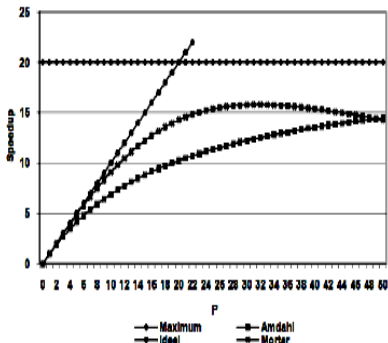
F=0.5

- every increment of S consumes new resources and is less rewarding:
- $S = 2 \Rightarrow 33\%$ speedup,
- $S = 5 \Rightarrow 6.67\%$ speedup.



Amdahl's Law: Parallel Speedup

$$S_P = \frac{T_1}{T_P} = \frac{P}{F + P(1-F)} < \frac{1}{1-F}$$



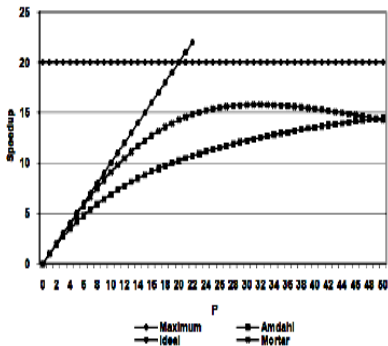
F=0.95

- Typically: speedup is sublinear, e.g., due to inter-thread communic.
- Sometimes superlinear speedup due to cache effects.
- Unforgiving Law: even if 99% is parallelized, $S_{\infty} < 100$.



Amdahl's Law: Parallel Speedup

$$S_P = \frac{T_1}{T_P} = \frac{P}{F + P(1-F)} < \frac{1}{1-F}$$



F=0.95

- Typically: speedup is sublinear, e.g., due to inter-thread communic.
- Sometimes superlinear speedup due to cache effects.
- Unforgiving Law: even if 99% is parallelized, $S_{\infty} < 100$.

Hardware Trend is to ever increase the number of cores.

Amdahl's Law: reason about parallelism asymptotically (∞ # cores), i.e., systematically exploit all levels of application's parallelism.



1 Implementation of Flat Bulk Operators

- Amdahl's Law
- Work-Depth Asymptotic
- Implementation of Reduce
- Implementation of Scan
- Implementation of Segmented Scan
- Other Second-Order Parallel Operators

2 Nested Data-Parallel Applications

- Sieve: Prime-Numbers Computation
- Nested Parallel Quicksort

3 Flattening Nested Parallelism

- Flattening Recipe (by Map Distribution)
- Re-Writing Rules For Flattening
- Flattening Prime-Number (Sieve) Computation
- Flattening Quicksort



Parallel Random Access Machine (PRAM)

PRAM focuses exclusively on parallelism and ignores issues related to synchronization and communication:

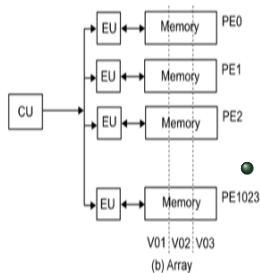
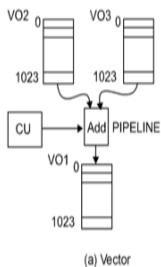
- 1 p processors connected to shared memory
- 2 each processor has an unique id (index) i , $1 \leq i \leq p$
- 3 SIMD execution, each parallel instruction requires unit time,
- 4 each processor has a flag that controls whether it is active in the execution of an instruction.



Parallel Random Access Machine (PRAM)

PRAM focuses exclusively on parallelism and ignores issues related to synchronization and communication:

- 1 p processors connected to shared memory
- 2 each processor has a unique id (index) i , $1 \leq i \leq p$
- 3 SIMD execution, each parallel instruction requires unit time,
- 4 each processor has a flag that controls whether it is active in the execution of an instruction.



• Work Time Algorithm (WT):

- **Work Complexity $W(n)$:** is the total # of ops performed,
- **Depth/Step Complexity $D(n)$:** is the # of sequential steps.

- If we know WT's work and depth, then Brent Theorem gives good complexity bounds for a PRAM.



1 Implementation of Flat Bulk Operators

- Amdahl's Law
- Work-Depth Asymptotic
- **Implementation of Reduce**
- Implementation of Scan
- Implementation of Segmented Scan
- Other Second-Order Parallel Operators

2 Nested Data-Parallel Applications

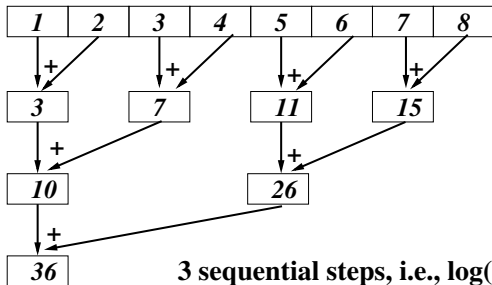
- Sieve: Prime-Numbers Computation
- Nested Parallel Quicksort

3 Flattening Nested Parallelism

- Flattening Recipe (by Map Distribution)
- Re-Writing Rules For Flattening
- Flattening Prime-Number (Sieve) Computation
- Flattening Quicksort



Reducing in Parallel



Reducing an array of length n with $n/2$ processors requires:

- work $W(n) = n$ and
- depth $D(n) = \lg n$, i.e., number of sequential steps.
- optimized runtime with P processors: $O(\frac{n}{P} + \lg P)$.

Theorem (Brent Theorem)

A Work-Time Algorithm of depth $D(n)$ and work $W(n)$ can be simulated on a P -processor PRAM in time complexity T such that:

$$\frac{W(n)}{P} \leq T < \frac{W(n)}{P} + D(n)$$



Reduce: Algorithm and Complexity

Input: array A of $n=2^k$ elems of type T
 $\oplus : T \times T \rightarrow T$ associative

Output: $S = \oplus_{j=1}^n a_j$

```

1. forall i = 0 to n-1 do
2.   B[i] ← A[i]
3. enddo

4. for h = 1 to k do
5.   forall i = 0 to n-1 by 2h do
6.     B[i] ← B[i] ⊕ B[i+2h-1]
7.   enddo
8. enddo
9. S ← B[0]
```

- $D_{1-3}(n) = \Theta(1)$, $W_{1-3}(n) = \Theta(n)$,
- $D_{5-7}(n) = \Theta(1)$,
 $W_{5-7}(n, h) = \Theta(n/2^h)$,
- $D_{4-8}(n) = k \times D_{5-7}(n) = \Theta(\lg n)$
- $W_{4-8}(n) = \sum_{h=1}^k W_{5-7}(n, h) = \Theta(\sum_{h=1}^k (n/2^h)) = \Theta(n)$
- $D_9(n) = \Theta(1)$, $W_9(n) = \Theta(1)$,
- $D(n) = \Theta(\lg n)$, $W(n) = \Theta(n)$!

$$O\left(\frac{n}{p}\right) \leq O(\text{Runtime}) < O\left(\frac{n}{p} + \lg n\right)$$

Note that the program is hardware-agnostic, i.e., has no notion of the number of cores.



Reduce: Naive Implementation in Futhark

— *Reduction by hand in Futhark: red-by-hand.fut*
 — *==*
 — *entry: futharkRed naiveRed*
 — *compiled input { [1.0f32, -2.0, 3.0, 1.0] }*
 — *output { 3.0f32 }*
 — *compiled random input { [33554432]f32 } auto output*

```
entry naiveRed [n] (a : [n]f32) : f32 = — assumes n = 2k
  let k = i64.f32 <| f32.log2 <| f32.i64 n
  let b =
    loop b = a for h < k do
      let n' = n >> (h+1)
      in map (\i -> b[2*i]+b[2*i+1]) (iota n')
  in b[0]
```

```
entry futharkRed [n] (a : [n]f32) : f32 =
  reduce (+) 0.0f32 a
```



Compiling and Profiling: Reduce Implem in Futhark

Performance w.r.t. the native reduce?

Benchmark with:

```
$ futhark bench —backend=cuda -e naiveRed red-by-hand.fut  
$ futhark bench —backend=cuda -e futharkRed red-by-hand.fut  
$ futhark bench —backend=cuda red-by-hand.fut
```

OR Compile and Run with (every command is on one line please):

```
$ futhark cuda red-by-hand.fut  
$ futhark dataset —f32-bounds=-1.0:1.0 -g [8388608]f32 -b |  
./red-by-hand —entry-point=naiveRed -t /dev/stderr -r 10
```

OR survey profile info with (every command is on one line please):

```
$ futhark bench —backend=cuda red-by-hand.fut  
-P —json=perf.json  
$ futhark profile perf.json  
$ more perf.prof/naiveRed/[33554432\]f32.summary
```



1 Implementation of Flat Bulk Operators

- Amdahl's Law
- Work-Depth Asymptotic
- Implementation of Reduce
- **Implementation of Scan**
- Implementation of Segmented Scan
- Other Second-Order Parallel Operators

2 Nested Data-Parallel Applications

- Sieve: Prime-Numbers Computation
- Nested Parallel Quicksort

3 Flattening Nested Parallelism

- Flattening Recipe (by Map Distribution)
- Re-Writing Rules For Flattening
- Flattening Prime-Number (Sieve) Computation
- Flattening Quicksort



Zip, Unzip, iota, replicate

- $\text{zip} : [n]\alpha_1 \rightarrow [n]\alpha_2 \rightarrow [n](\alpha_1, \alpha_2)$
- $\text{zip } [a_1, \dots, a_n] [b_1, \dots, b_n] \equiv [(a_1, b_1), \dots, (a_n, b_n)],$



Zip, Unzip, iota, replicate

- $\text{zip} : [n]\alpha_1 \rightarrow [n]\alpha_2 \rightarrow [n](\alpha_1, \alpha_2)$
- $\text{zip } [a_1, \dots, a_n] [b_1, \dots, b_n] \equiv [(a_1, b_1), \dots, (a_n, b_n)],$
- $\text{unzip} : [n](\alpha_1, \alpha_2) \rightarrow ([n]\alpha_1, [n]\alpha_2)$
- $\text{unzip } [(a_1, b_1), \dots, (a_n, b_n)] \equiv ([a_1, \dots, a_n], [b_1, \dots, b_n]),$
- In some sense zip/unzip are syntactic sugar



Zip, Unzip, iota, replicate

- $\text{zip} : [n]\alpha_1 \rightarrow [n]\alpha_2 \rightarrow [n](\alpha_1, \alpha_2)$
- $\text{zip } [a_1, \dots, a_n] [b_1, \dots, b_n] \equiv [(a_1, b_1), \dots, (a_n, b_n)],$
- $\text{unzip} : [n](\alpha_1, \alpha_2) \rightarrow ([n]\alpha_1, [n]\alpha_2)$
- $\text{unzip } [(a_1, b_1), \dots, (a_n, b_n)] \equiv ([a_1, \dots, a_n], [b_1, \dots, b_n]),$
- In some sense zip/unzip are syntactic sugar
- $\text{replicate} : (n: \text{int}) \rightarrow \alpha \rightarrow [n]\alpha$
- $\text{replicate } n \ a \equiv [a, a, \dots, a],$



Zip, Unzip, iota, replicate

- $\text{zip} : [n]\alpha_1 \rightarrow [n]\alpha_2 \rightarrow [n](\alpha_1, \alpha_2)$
- $\text{zip } [a_1, \dots, a_n] [b_1, \dots, b_n] \equiv [(a_1, b_1), \dots, (a_n, b_n)],$
- $\text{unzip} : [n](\alpha_1, \alpha_2) \rightarrow ([n]\alpha_1, [n]\alpha_2)$
- $\text{unzip } [(a_1, b_1), \dots, (a_n, b_n)] \equiv ([a_1, \dots, a_n], [b_1, \dots, b_n]),$
- In some sense zip/unzip are syntactic sugar
- $\text{replicate} : (n: \text{int}) \rightarrow \alpha \rightarrow [n]\alpha$
- $\text{replicate } n \ a \equiv [a, a, \dots, a],$
- $\text{iota} : (n: \text{int}) \rightarrow [n]\text{int}$
- $\text{iota } n \equiv [0, 1, \dots, n-1]$

Note: in Haskell zip does not expect same-length arrays;
in Futhark it does!

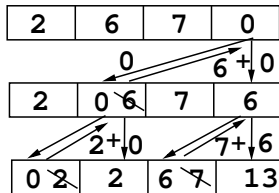
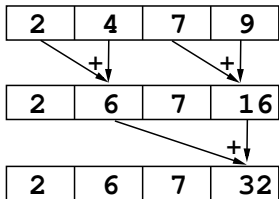


Map, Reduce, and Scan Types and Semantics

- $[n]\alpha$ denotes the type of an array of n elements of type α .
- $\text{map} : (\alpha \rightarrow \beta) \rightarrow [n]\alpha \rightarrow [n]\beta$
 $\text{map } f \ [x_1, \dots, x_n] = [f \ x_1, \dots, f \ x_n],$
 i.e., $x_i : \alpha, \forall i$, and $f : \alpha \rightarrow \beta$.
- $\text{reduce} : (\alpha \rightarrow \alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow [n]\alpha \rightarrow \alpha$
 $\text{reduce } \odot \ e \ [x_1, x_2, \dots, x_n] = e \odot x_1 \odot x_2 \odot \dots \odot x_n,$
 i.e., $e : \alpha$, $x_i : \alpha, \forall i$, and $\odot : \alpha \rightarrow \alpha \rightarrow \alpha$.
- $\text{scan}^{\text{exc}} : (\alpha \rightarrow \alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow [n]\alpha \rightarrow [n]\alpha$
 $\text{scan}^{\text{exc}} \odot \ e \ [x_1, \dots, x_n] = [e, e \odot x_1, \dots, e \odot x_1 \odot \dots \odot x_{n-1}]$
 i.e., $e : \alpha$, $x_i : \alpha, \forall i$, and $\odot : \alpha \rightarrow \alpha \rightarrow \alpha$.
- $\text{scan}^{\text{inc}} : (\alpha \rightarrow \alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow [n]\alpha \rightarrow [n]\alpha$
 $\text{scan}^{\text{inc}} \odot \ e \ [x_1, \dots, x_n] = [e \odot x_1, \dots, e \odot x_1 \odot \dots \odot x_n]$
 i.e., $e : \alpha$, $x_i : \alpha, \forall i$, and $\odot : \alpha \rightarrow \alpha \rightarrow \alpha$.



Parallel Exclusive Scan with Associative Operator $\oplus$



Up-Sweep & Down-Sweep

Two Steps:

- **Up-Sweep:** similar with reduction
- Root is replaced with neutral element.
- **Down-Sweep:**
 - the left child sends its value to parent and updates its value to that of parent.
 - the right-child value is given by $\oplus$ applied to the left-child value and the (old) value of parent.
 - note that the right child is in fact the parent, i.e., in-place algorithm.



Parallel Exclusive Scan Algorithm And Complexity

Input: array A of $n=2^k$ elems of type T

$\oplus :: T \times T \rightarrow T$ associative

Output: $B = [0, a_1, a_1 \oplus a_2, \dots, \oplus_{j=1}^{n-1} a_j]$

```

1. forall i = 0 : n-1 do
2.   B[i] ← A[i]
3. enddo

4. for d = 0 to k-1 do // up-sweep
5.   forall i = 0 to n-1 by  $2^{d+1}$  do
6.     B[i+ $2^{d+1}$ -1] ← B[i+ $2^d$ -1]  $\oplus$ 
                        B[i+ $2^{d+1}$ -1]
7.   enddo
8. enddo
9. B[n-1] = 0
10. for d = k-1 downto 0 do // down-sweep
11.   forall i = 0 to n-1 by  $2^{d+1}$  do
12.     tmp ← B[i+ $2^d$ -1]
13.     B[i+ $2^d$ -1] ← B[i+ $2^{d+1}$ -1]
14.     B[i+ $2^{d+1}$ -1] ← tmp  $\oplus$  B[i+ $2^{d+1}$ -1]
15.   enddo
16. enddo

```

- The code show exponentials for clarity, but those can be computed by one multiplication/division operation each sequential iteration.
- $D(n) = \Theta(\lg n)$, $W(n) = \Theta(n)$!
- Similar reasoning as with reduce.



1 Implementation of Flat Bulk Operators

- Amdahl's Law
- Work-Depth Asymptotic
- Implementation of Reduce
- Implementation of Scan
- **Implementation of Segmented Scan**
- Other Second-Order Parallel Operators

2 Nested Data-Parallel Applications

- Sieve: Prime-Numbers Computation
- Nested Parallel Quicksort

3 Flattening Nested Parallelism

- Flattening Recipe (by Map Distribution)
- Re-Writing Rules For Flattening
- Flattening Prime-Number (Sieve) Computation
- Flattening Quicksort



Segmented Inclusive Scan with Operator $\oplus$

Flat representation of 2D iregular arrays (arrays-of-arrays):

- Flat Data: a 1D array containing the data in flat format.
- Flag Array: a 1D array containing a 1 at the start position of each subarray, and 0 otherwise.
- Example iregular array: $[[1,2,3], [4], [5,6,7,8,9]]$, shape: $[3,1,5]$
Flat Data: $[1,2,3,4,5,6,7,8,9]$
Flag Array: $[1,0,0,1,1,0,0,0,0]$



Segmented Inclusive Scan with Operator $\oplus$

Flat representation of 2D irregular arrays (arrays-of-arrays):

- Flat Data: a 1D array containing the data in flat format.
- Flag Array: a 1D array containing a 1 at the start position of each subarray, and 0 otherwise.
- Example irregular array: $[[1,2,3], [4], [5,6,7,8,9]]$, shape: $[3,1,5]$
 Flat Data: $[1,2,3,4,5,6,7,8,9]$
 Flag Array: $[1,0,0,1,1,0,0,0,0]$

Segmented Scan is Equivalent with Mapping a Scan op on each subarray of an irregular 2D array; hence it is a shape preserving op.

```
map (\a -> scan (+) 0 a)
  [[1,2,3], [1,2,3,4]]
[ scaninc (+) 0 [1,2,3],
  scaninc (+) 0 [1,2,3,4] ]
≡
[ [1,3,6], [1,3,6,10] ]
```

```
-- Flags & Flat Data Representation:
sgmScanInc (+) 0 [1,0,0,1,0,0,0] -- flag
                    [1,2,3,1,2,3,4] -- data
≡
[1,3,6,1,3,6,10] -- scanned data
```



Segmented Inclusive Scan with Operator $\oplus$ (Haskell)

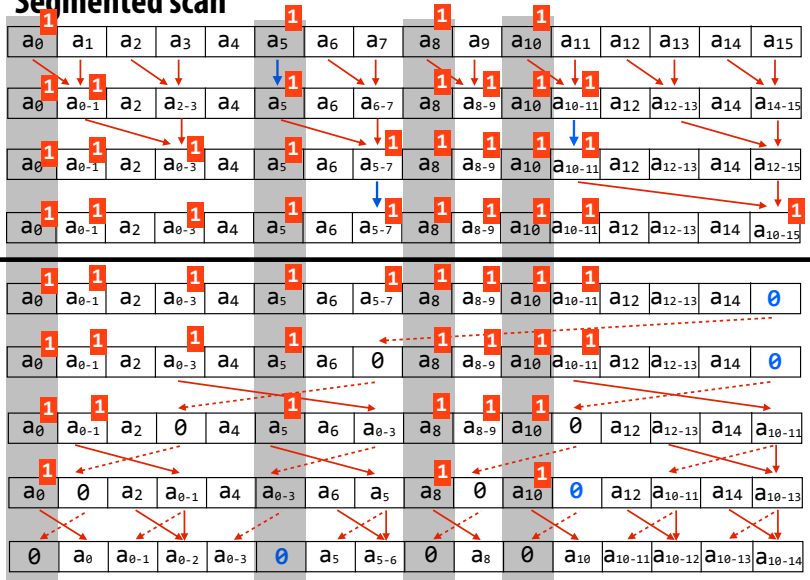
```
map (\a -> scan (+) 0 a)
    [[1,2,3],[1,2,3,4]]     $\equiv$ 
```

```
-- Flags & Flat Data Representation:
sgmScanInc (+) 0 [1,0,0,1,0,0,0] -- flag
                                [1,2,3,1,2,3,4] -- data
```



Slide from CMU 15-418: Parallel Computer Architecture and Programming (Spring 2012)

Segmented scan



Segmented Exclusive Scan Alg And Complexity

Input: flag array F of $n=2^k$ of ints
 data array A of $n=2^k$ elems of type T
 $\oplus :: T \times T \rightarrow T$ associative

Output: B = segmented scan of 2-dim array A

```

1.  FORALL i = 0 to n-1 do B[i] ← A[i] ENDDO
2.  FOR d = 0 to k-1 DO // up-sweep
3.    FORALL i = 0 to n-1 by 2d+1 DO
4.      IF F[i+2d+1-1] == 0 THEN
5.        B[i+2d+1-1] ← B[i+2d-1] ⊕ B[i+2d+1-1]
6.      ENDIF
7.      F[i+2d+1-1] ← F[i+2d-1] .|. F[i+2d+1-1]
8.    ENDDO ENDDO
9.  B[n-1] ← 0
10. FOR d = k-1 downto 0 DO // down-sweep
11.   FORALL i = 0 to n-1 by 2d+1 DO
12.     tmp ← B[i+2d-1]
13.     IF Foriginal[i+2d] ≠ 0 THEN
14.       B[i+2d+1-1] ← 0
15.     ELSE IF F[i+2d-1] ≠ 0 THEN
16.       B[i+2d+1-1] ← tmp
17.     ELSE B[i+2d+1-1] ← tmp ⊕ B[i+2d+1-1]
18.     ENDIF
19.   F[i+2d+1-1] ← 0
20. ENDDO ENDDO
  
```

- While there are more branches, the asymptotics does not change:
- $D(n) = \Theta(\lg n)$,
 $W(n) = \Theta(n)!$



Segmented Inclusive Scan with Operator $\oplus$

Equiv with Mapping a Scan op on each segment of an irregular array.

```
map (\a -> scan (+) 0 a)
    [[1,2,3],[1,2,3,4]]
    ≡
[ scaninc (+) 0 [1,2,3],
  scaninc (+) 0 [1,2,3,4] ]
    ≡
[ [1,3,6], [1,3,6,10] ]
```

```
-- Flags & Flat Data Representation:
sgmScanInc (+) 0 [1,0,0,1,0,0,0] -- flag
                        [1,2,3,1,2,3,4] -- data
    ≡
[1,3,6,1,3,6,10]      -- scanned data
```



Segmented Inclusive Scan with Operator $\oplus$

Equiv with Mapping a Scan op on each segment of an irregular array.

```
map (\a -> scan (+) 0 a)
  [[1,2,3],[1,2,3,4]]
  ≡
[ scaninc (+) 0 [1,2,3],
  scaninc (+) 0 [1,2,3,4] ]
  ≡
[ [1,3,6], [1,3,6,10] ]
```

```
-- Flags & Flat Data Representation:
sgmScanInc (+) 0 [1,0,0,1,0,0,0] -- flag
                    [1,2,3,1,2,3,4] -- data
  ≡
[1,3,6,1,3,6,10] -- scanned data
```

Can be obtained by replacing the following Futhark operator:

```
let segmented_scan [n] 't (op: t -> t -> t) (ne: t)
  (flags: [n] bool) (arr: [n] t) : [n] t =
  let (_, res) = unzip <|
    scan (\(x_flag, x) (y_flag, y) ->

      let fl = ???
      let vl = ???

      in (fl, vl)
    ) (false, ne) (zip flags arr)
  in res
```



Segmented Inclusive Scan with Operator $\oplus$

Can be obtained by replacing the following Futhark operator:

```

let segmented_scan [n] 't
    (op: t  $\rightarrow$  t  $\rightarrow$  t) (ne: t)
    (flags: [n] bool)
    (arr: [n] t) : [n] t =
  let (_, res) = unzip <|

    scan (\(x_flag, x) (y_flag, y)  $\rightarrow$ 
      let fl = x_flag || y_flag
      let vl = if y_flag then y
              else op x y
      in (fl, vl)
    ) (false, ne) (zip flags arr)

  in res

```



1 Implementation of Flat Bulk Operators

- Amdahl's Law
- Work-Depth Asymptotic
- Implementation of Reduce
- Implementation of Scan
- Implementation of Segmented Scan
- Other Second-Order Parallel Operators

2 Nested Data-Parallel Applications

- Sieve: Prime-Numbers Computation
- Nested Parallel Quicksort

3 Flattening Nested Parallelism

- Flattening Recipe (by Map Distribution)
- Re-Writing Rules For Flattening
- Flattening Prime-Number (Sieve) Computation
- Flattening Quicksort



Map2, Filter

- $\text{map2} : (\alpha_1 \rightarrow \alpha_2 \rightarrow \beta) \rightarrow [\mathbf{n}] \alpha_1 \rightarrow [\mathbf{n}] \alpha_2 \rightarrow [\mathbf{n}] \beta$
- $\text{map2} \odot [a_1, \dots, a_n] [b_1, \dots, b_n] \equiv [a_1 \odot b_1, \dots, a_n \odot b_n]$
- $\text{map3} \dots$



Map2, Filter

- $\text{map2} : (\alpha_1 \rightarrow \alpha_2 \rightarrow \beta) \rightarrow [\mathbf{n}]\alpha_1 \rightarrow [\mathbf{n}]\alpha_2 \rightarrow [\mathbf{n}]\beta$
- $\text{map2} \odot [a_1, \dots, a_n] [b_1, \dots, b_n] \equiv [a_1 \odot b_1, \dots, a_n \odot b_n]$
- $\text{map3} \dots$
- $\text{filter} : (\alpha \rightarrow \text{Bool}) \rightarrow [\mathbf{n}]\alpha \rightarrow [\mathbf{m}]\alpha$ (where $m \leq n$)
- $\text{filter } p [a_1, \dots, a_n] = [a_{k_1}, \dots, a_{k_m}]$ such that $k_1 < k_2 < \dots < k_m$, and denoting by $\bar{k} = k_1, \dots, k_m$, we have $(p \ a_j == \text{true}) \ \forall j \in \bar{k}$, **and** $(p \ a_j == \text{false}) \ \forall j \notin \bar{k}$.

Note: in Haskell `map2`, `map3` do not expect same-length arrays; in Futhark they do!



Filter (Blank)



Scatter: A Parallel Write Operator

Scatter **updates in parallel** a base array with a set of values at specified indices:

$\text{scatter} : *[m]\alpha \rightarrow [n]\text{int} \rightarrow [n]\alpha \rightarrow *[m]\alpha$

A (data vector) = [b0, b1, b2, b3]

I (index vector) = [2, 4, 1, -1]

X (input array) = [a0, a1, a2, a3, a4, a5]

scatter X I A = [a0, b2, b0, a3, b1, a5]



Scatter: A Parallel Write Operator

Scatter updates in parallel a base array with a set of values at specified indices:

$\text{scatter} : *[m]\alpha \rightarrow [n]\text{int} \rightarrow [n]\alpha \rightarrow *[m]\alpha$

A (data vector) = [b0, b1, b2, b3]

I (index vector) = [2, 4, 1, -1]

X (input array) = [a0, a1, a2, a3, a4, a5]

$\text{scatter } X \text{ I } A = [a0, b2, b0, a3, b1, a5]$

scatter has $D(n) = \Theta(1)$ and $W(n) = \Theta(n)$,

i.e., requires n update operations (n is the size of I or A , not of X !).

- 1 Array X is consumed by scatter; following uses of X are illegal!
- 2 Similarly, X can alias neither I nor A !

In Futhark, scatter checks and ignores the indices that are out of bounds (no update is performed on those). This is useful for padding the iteration space in order to obtain regular parallelism.



Permute, Split, Replicate, Iota

- Operator to permute in parallel based on a set (array) of indices:
 $\text{permute} : [n]\text{int} \rightarrow [n]\alpha \rightarrow [n]\alpha.$

$\text{permute } I \ A \equiv \text{scatter } (\text{replicate } n \ e) \ I \ A$

A (data vector) = [a0, a1, a2, a3, a4, a5]

I (index vector) = [3, 2, 0, 4, 1, 5]

$\text{permute } I \ A = [a2, a4, a1, a0, a3, a5]$

- $\text{split} : (i:\text{int}) \rightarrow [n]\alpha \rightarrow ([i]\alpha, [n-i]\alpha)$
 $\text{split } i \ [a_0, \dots, a_{n-1}] \equiv ([a_0, \dots, a_{i-1}], [a_i, \dots, a_{n-1}])$
- $\text{replicate} : (n:\text{int}) \rightarrow \alpha \rightarrow [n]\alpha$
 $\text{replicate } n \ a \equiv [a, a, \dots, a]$, i.e., a is replicated n times.
- $\text{iota} : (n:\text{int}) \rightarrow [n]\text{int}$
 $\text{iota } n = [0, \dots, n-1]$



Partition2: Plan for the indices of "true" elements

`partition2` : $(\alpha \rightarrow \text{Bool}) \rightarrow [n]\alpha \rightarrow (i64, [n]\alpha)$

In the result, elements satisfying the predicate occur before the others

The plan is to compute the indices in the result for each input element, then to apply `scatter`. Assume input array $X = [5, 4, 2, 3, 7, 8]$ and a predicate that succeeds for even numbers. The result array should be: $[4, 2, 8, 5, 3, 7]$.

How to compute the indices `isT` of the elements that succeed?

- We have: $X = [5, 4, 2, 3, 7, 8]$
- We want: $\text{isT} = [x, 0, 1, x, x, 2]$



Partition2: Plan for the indices of "true" elements

`partition2` : $(\alpha \rightarrow \text{Bool}) \rightarrow [n]\alpha \rightarrow (i64, [n]\alpha)$

In the result, elements satisfying the predicate occur before the others

The plan is to compute the indices in the result for each input element, then to apply `scatter`. Assume input array $X = [5, 4, 2, 3, 7, 8]$ and a predicate that succeeds for even numbers. The result array should be: $[4, 2, 8, 5, 3, 7]$.

How to compute the indices `isT` of the elements that succeed?

- We have: $X = [5, 4, 2, 3, 7, 8]$
- We want: $\text{isT} = [x, 0, 1, x, x, 2]$

(1) map the predicate on X and coerce to ints:

$\text{cs} = [0, 1, 1, 0, 0, 1]$



Partition2: Plan for the indices of "true" elements

`partition2` : $(\alpha \rightarrow \text{Bool}) \rightarrow [n]\alpha \rightarrow (i64, [n]\alpha)$

In the result, elements satisfying the predicate occur before the others

The plan is to compute the indices in the result for each input element, then to apply `scatter`. Assume input array $X = [5, 4, 2, 3, 7, 8]$ and a predicate that succeeds for even numbers. The result array should be: $[4, 2, 8, 5, 3, 7]$.

How to compute the indices `isT` of the elements that succeed?

- We have: $X = [5, 4, 2, 3, 7, 8]$
- We want: $\text{isT} = [x, 0, 1, x, x, 2]$

(1) map the predicate on X and coerce to ints:

$\text{cs} = [0, 1, 1, 0, 0, 1]$

(2) perform an inclusive scan on cs

$\text{is} = [0, 1, 2, 2, 2, 3]$



Partition2: Plan for the indices of "true" elements

`partition2` : $(\alpha \rightarrow \text{Bool}) \rightarrow [n]\alpha \rightarrow (i64, [n]\alpha)$

In the result, elements satisfying the predicate occur before the others

The plan is to compute the indices in the result for each input element, then to apply scatter. Assume input array $X = [5, 4, 2, 3, 7, 8]$ and a predicate that succeeds for even numbers. The result array should be: $[4, 2, 8, 5, 3, 7]$.

How to compute the indices isT of the elements that succeed?

- We have: $X = [5, 4, 2, 3, 7, 8]$
- We want: $\text{isT} = [x, 0, 1, x, x, 2]$

(1) map the predicate on X and coerce to ints:

$$cs = [0, 1, 1, 0, 0, 1]$$

(2) perform an inclusive scan on cs

$$is = [0, 1, 2, 2, 2, 3]$$

(3) subtract one from each element:

$$\text{isT} = [-1, 0, 1, 1, 1, 2]$$

- steps (2) & (3) equivalent with an exclusive scan.



Partition2: Plan for the indices of "false" elements

`partition2` : $(\alpha \rightarrow \text{Bool}) \rightarrow [n]\alpha \rightarrow (i64, [n]\alpha)$

In the result, elements satisfying the predicate occur before the others

The plan is to compute the indices in the result for each input element, then to apply `scatter`. Assume input array $X = [5, 4, 2, 3, 7, 8]$ and a predicate that succeeds for even numbers. The result array should be: $[4, 2, 8, 5, 3, 7]$.

How to compute the indices `isF` of the elements that fail?

- We have: $X = [5, 4, 2, 3, 7, 8]$
- We want: $\text{isF} = [3, x, x, 4, 5, x]$

(1) map the negation of the predicate on X and coerce to ints:

$\text{cs} = [1, 0, 0, 1, 1, 0]$



Partition2: Plan for the indices of "false" elements

`partition2` : $(\alpha \rightarrow \text{Bool}) \rightarrow [n]\alpha \rightarrow (i64, [n]\alpha)$

In the result, elements satisfying the predicate occur before the others

The plan is to compute the indices in the result for each input element, then to apply `scatter`. Assume input array $X = [5, 4, 2, 3, 7, 8]$ and a predicate that succeeds for even numbers. The result array should be: $[4, 2, 8, 5, 3, 7]$.

How to compute the indices `isF` of the elements that fail?

- We have: $X = [5, 4, 2, 3, 7, 8]$
- We want: $\text{isF} = [3, x, x, 4, 5, x]$

(1) map the negation of the predicate on X and coerce to ints:

$\text{cs} = [1, 0, 0, 1, 1, 0]$

(2) perform an inclusive scan on cs

$\text{is} = [1, 1, 1, 2, 3, 3]$



Partition2: Plan for the indices of "false" elements

`partition2` : $(\alpha \rightarrow \text{Bool}) \rightarrow [n]\alpha \rightarrow (i64, [n]\alpha)$

In the result, elements satisfying the predicate occur before the others

The plan is to compute the indices in the result for each input element, then to apply scatter. Assume input array $X = [5, 4, 2, 3, 7, 8]$ and a predicate that succeeds for even numbers. The result array should be: $[4, 2, 8, 5, 3, 7]$.

How to compute the indices `isF` of the elements that fail?

- We have: $X = [5, 4, 2, 3, 7, 8]$
- We want: $\text{isF} = [3, x, x, 4, 5, x]$

(1) map the negation of the predicate on X and coerce to ints:

$\text{cs} = [1, 0, 0, 1, 1, 0]$

(2) perform an inclusive scan on cs

$\text{is} = [1, 1, 1, 2, 3, 3]$

(3) add $s - 1$ to each element, where $s = 3$ is the split point:

$\text{isF} = [3, 3, 3, 4, 5, 6]$



Partition2/Filter Implementation

`partition2` : $(\alpha \rightarrow \text{Bool}) \rightarrow [n]\alpha \rightarrow (\text{i64}, [n]\alpha)$

In result, the elements satisfying the predicate occur before the others.

Can be implemented by means of `map`, `scan` and `scatter`.



Partition2/Filter Implementation

`partition2` : $(\alpha \rightarrow \text{Bool}) \rightarrow [n]\alpha \rightarrow (i64, [n]\alpha)$

In result, the elements satisfying the predicate occur before the others.

Can be implemented by means of `map`, `scan` and `scatter`.

```
let partition2 't [n] (dummy: t)
    (pred: t -> bool) (X: [n]t) :
    (i64, [n]t) =
```

Assume $X = [5,4,2,3,7,8]$, and
pred is T(rue) for even nums.

```
let cs = map pred X
let tfs= map (\ f->if f then 1
                else 0) cs

let isT= scan (+) 0 tfs
let s  = if n==0 then 0 else isT[n-1]
```

```
let ffs= map (\ f->if f then 0
                else 1) cs

let isF= map (+s) <| scan (+) 0 ffs
let inds=map (\(c,iT,iF) ->
                if c then iT-1
                else iF-1
            ) (zip3 cs isT isF)
```

```
let tmp = replicate n dummy
in (s, scatter tmp inds X)
```



Partition2/Filter Implementation

`partition2` : $(\alpha \rightarrow \text{Bool}) \rightarrow [n]\alpha \rightarrow (i64, [n]\alpha)$

In result, the elements satisfying the predicate occur before the others.

Can be implemented by means of `map`, `scan` and `scatter`.

```
let partition2 't [n] (dummy: t)
    (pred: t -> bool) (X: [n]t) :
    (i64, [n]t) =

    let cs = map pred X
    let tfs= map (\ f->if f then 1
                      else 0) cs

    let isT= scan (+) 0 tfs
    let s  = if n==0 then 0 else isT[n-1]

    let ffs= map (\ f->if f then 0
                      else 1) cs
    let isF= map (+s) <| scan (+) 0 ffs
    let inds=map ((c,iT,iF) ->
                  if c then iT-1
                  else iF-1
                  ) (zip3 cs isT isF)

    let tmp = replicate n dummy
    in (s, scatter tmp inds X)
```

Assume $X = [5,4,2,3,7,8]$, and
pred is T(rue) for even nums.

```
n      = 6
cs     = [F, T, T, F, F, T]
tfs    = [0, 1, 1, 0, 0, 1]
```

```
isT    = [0, 1, 2, 2, 2, 3]
i      = 3
```

```
ffs    = [1, 0, 0, 1, 1, 0]
isF    = [4, 4, 4, 5, 6, 6]
```

```
inds= [3, 0, 1, 4, 5, 2]
```

```
flags  = [3, 0, 0, 3, 0, 0]
Result = [4, 2, 8, 5, 3, 7]
```



1 Implementation of Flat Bulk Operators

- Amdahl's Law
- Work-Depth Asymptotic
- Implementation of Reduce
- Implementation of Scan
- Implementation of Segmented Scan
- Other Second-Order Parallel Operators

2 Nested Data-Parallel Applications

- Sieve: Prime-Numbers Computation
- Nested Parallel Quicksort

3 Flattening Nested Parallelism

- Flattening Recipe (by Map Distribution)
- Re-Writing Rules For Flattening
- Flattening Prime-Number (Sieve) Computation
- Flattening Quicksort



Computing Prime Numbers up To n : First Attempt

See also "Scan as Primitive Parallel Operation" [Blelloch].

Start with an array of size n filled initially with 1, i.e., all are primes, and iteratively zero out all multiples of numbers up to $\sqrt{n}$.

```
int res[n] = {0, 0, 1, 1, 1, ..., 1}
for(i = 2; i <= sqrt(n); i++) { //sequential
    if ( res[i] != 0 ) {
        forall m  $\in$  multiples of  $i \leq n$  do {
            res[m] = 0;
        }
    }
}
```

Work: $O(n \lg \lg n)$ but Depth: $O(\sqrt{n})$ (Not Good Enough!)

Why $i \leq \text{sqrt}(n)$?



Computing Prime Numbers up To n : First Attempt

See also "Scan as Primitive Parallel Operation" [Blelloch].

Start with an array of size n filled initially with 1, i.e., all are primes, and iteratively zero out all multiples of numbers up to $\sqrt{n}$.

```
int res[n] = {0, 0, 1, 1, 1, ..., 1}
for(i = 2; i <= sqrt(n); i++) { //sequential
    if ( res[i] != 0 ) {
        forall m  $\in$  multiples of  $i \leq n$  do {
            res[m] = 0;
        }
    }
}
```

Work: $O(n \lg \lg n)$ but Depth: $O(\sqrt{n})$ (Not Good Enough!)

Why $i \leq \text{sqrt}(n)$?

Because a non-prime number has to have a multiple less than $\sqrt{n}$.



Sieve: Illustrating the Computation for $n = 9$

Indices: 0 1 2 3 4 5 6 7 8 9

Initially: $\text{res} = [0, 0, 1, 1, 1, 1, 1, 1, 1, 1]$

Iter $i = 2$ zeroes out multiples of 2

$\text{res} = [0, 0, 1, 1, 0, 1, 0, 1, 0, 1]$

Iter $i = 3$ zeroes out multiples of 3

$\text{res} = [0, 0, 1, 1, 0, 1, 0, 1, 0, 0]$

Loop ends

The prime numbers are the indices of the 1 elements: $\{2, 3, 5, 7\}$



Computing Prime Numbers: 1st Attempt (Futhark)

Start with an array of size n filled initially with 1, i.e., all are primes, and iteratively zero out all multiples of numbers up to $\sqrt{n}$.

```
let primesHelp [np1] (sq : i64)
  (a : *[np1]i32) : [np1]i32 =
  let n = np1 - 1 in
  loop(a) for j < (sq-1) do
    let i = j + 2
    let m = (n / i) - 1
    let inds = map (\k->(k+2)*i)(iota m)
    in scatter a inds (replicate m 0)
```

```
let main (n : i64) : []i64 =
  let a = map (\i->if i==0 || i==1
                    then 0 else 1)
            (iota (n+1))
  let sq = i64.f64 (f64.sqrt (f64.i64 n))
  let fl = primesHelp sq a
  in filter (\i->#[unsafe] fl[i]!=0)
            (iota (n+1))
```

Assume $n = 9$, $\text{sqrt}N = 3$
 $a = [0,0,1,1,1,1,1,1,1]$

iteration $j = 0$, $i = 2$
 $m = (9 \text{ 'div' } 2) - 1 = 3$
 $\text{inds} = [4, 6, 8]$
 $\text{vals} = [0, 0, 0]$
 $a' = [0,0,1,1,0,1,0,1,0]$

iteration $j = 1$, $i = 3$
 $m = (9 \text{ 'div' } 3) - 1 = 2$
 $\text{inds} = [6, 9]$
 $\text{vals} = [0, 0]$
 $a'' = [0,0,1,1,0,1,0,1,0]$

iteration $j = 2$: loop ends.
 result: $[0,0,1,1,0,1,0,1,0]$
 after filter: $[2,3,5,7]$

Work: $O(n \lg \lg n)$ but Depth: $O(\sqrt{n})$ (Not Good Enough!)



Prime Numbers: Nested Parallelism in Haskell

If we have all primes from 2 to $\sqrt{n}$ we could generate all multiples of these primes at once: $\{[2*p:n:p]: p \text{ in } \text{sqr_primes}\}$ in NESL.

Also call algorithm recursively on $\sqrt{n} \Rightarrow \text{Depth: } O(\lg \lg n)!$

(solution of $n^{(1/2)^{\text{depth}}} = 2$).



Prime Numbers: Nested Parallelism in Haskell

If we have all primes from 2 to $\sqrt{n}$ we could generate all multiples of these primes at once: $\{[2*p:n:p] : p \text{ in } \text{sqr_primes}\}$ in NESL.

Also call algorithm recursively on $\sqrt{n} \Rightarrow \text{Depth: } O(\lg \lg n)!$

(solution of $n^{(1/2)^{\text{depth}}} = 2$).

```
primesOpt :: Int -> [Int]
primesOpt n =
  if n <= 2 then [2]
  else
    let sqrtN = floor (sqrt (fromIntegral n))
        sqr_primes = primesOpt sqrtN
        nested = map (\p->let m = (n `div` p)
                        in map (\j-> j*p)
                           [2..m]
                   ) sqr_primes
        not_primes = reduce (++) [] nested
        mm = length not_primes
        zeros = replicate mm False
        prime_flags=scatter(replicate (n+1) True)
                        not_primes zeros
        (primes,_)= unzip $ filter (\(i,f)->f)
                        $ (zip [0..n] prime_flags)
    in drop 2 primes
```



Prime Numbers: Nested Parallelism in Haskell

If we have all primes from 2 to $\sqrt{n}$ we could generate all multiples of these primes at once: $\{[2*p:n:p] : p \text{ in } \text{sqr_primes}\}$ in NESL.

Also call algorithm recursively on $\sqrt{n} \Rightarrow \text{Depth: } O(\lg \lg n)!$

(solution of $n^{(1/2)^{\text{depth}}} = 2$).

```
primesOpt :: Int -> [Int]
primesOpt n =
  if n <= 2 then [2]
  else
    let sqrtN = floor (sqrt (fromIntegral n))
        sqrt_primes = primesOpt sqrtN
        nested = map (\p->let m = (n `div` p)
                          in map (\j-> j*p)
                                [2..m]
                      ) sqrt_primes
        not_primes = reduce (++) [] nested
        mm = length not_primes
        zeros = replicate mm False
        prime_flags=scatter(replicate (n+1) True) not_primes zeros
        (primes,_)= unzip $ filter (\(i,f)->f)
                          $ (zip [0..n] prime_flags)
    in drop 2 primes
```

Assume $n = 9$, $\text{sqrtN} = 3$

```
call primesOpt 3
n = 3, sqrtN = 1, sqrt_primes=[2]
nested = [[]]; not_primes = []
mm = 0; zeros = []
prime_flags = [T,T,T,T]
primes = [0,1,2,3]; returns [2,3]

in primesOpt 9, after
return from primesOpt3,
sqrt_primes = [2,3]
nested = [[4,6,8],[6,9]]
not_primes = [4,6,8,6,9]
mm=5; zeros= [F,F,F,F,F]
prime_flags= [T,T,T,T,F,T,F,T,F,F]
primes = [0,1,2,3,5,7]
returns [2,3,5,7]
```



Quicksort with Nested Parallelism

```
nestedQuicksort :: [a] -> [a]
nestedQuicksort arr =
  if (length arr) <= 1 then arr else
  let i = getRand (0, (length arr) - 1)
      a = arr !! i
      s1 = filter (\x -> (x < a)) arr
      s2 = filter (\x -> (x >= a)) arr
      rs = map nestedQuicksort [s1, s2]
  in  (rs !! 0) ++ (rs !! 1)
```

- Is this implementation correct?
- Average Depth and Work ?



Quicksort with Nested Parallelism

```
nestedQuicksort :: [a] -> [a]
nestedQuicksort arr =
  if (length arr) <= 1 then arr else
    let i = getRand (0, (length arr) - 1)
        a = arr !! i
        s1 = filter (\x -> (x < a)) arr
        s2 = filter (\x -> (x >= a)) arr
        rs = map nestedQuicksort [s1, s2]
    in  (rs !! 0) ++ (rs !! 1)
```

```
-- Is this implementation correct?
-- Average Depth and Work ?
```

Assume input array [3,2,4,1]
 Assume random $i = 0 \Rightarrow a = 3$

```
s1 = [2,1]
s2 = [3,4]
```

```
nestedQuicksort [2,1]:
i = 0, a = 2
s1 = [1]
s2 = [2]
results in [1]++[2]==[1,2]
```

```
nestedQuicksort [3,4]: ...
results in [3,4]
```

```
After recursion concat:
[1,2] ++ [3,4] = [1,2,3,4]
```

Denoting by n the size of the input array: Average Work is $O(n \lg N)$.

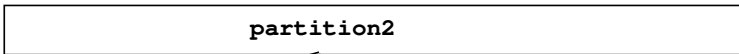
If filter would have depth 1, then Average Depth: $O(\lg n)$.

In practice we have depth: $O(\lg^2 n)$.



Quicksort: Illustrating Flat-Parallel Execution

Macro
Step 0



Macro
Step 1



Macro
Step 2



.....



- 1 Implementation of Flat Bulk Operators
 - Amdahl's Law
 - Work-Depth Asymptotic
 - Implementation of Reduce
 - Implementation of Scan
 - Implementation of Segmented Scan
 - Other Second-Order Parallel Operators
- 2 Nested Data-Parallel Applications
 - Sieve: Prime-Numbers Computation
 - Nested Parallel Quicksort
- 3 Flattening Nested Parallelism
 - Flattening Recipe (by Map Distribution)
 - Re-Writing Rules For Flattening
 - Flattening Prime-Number (Sieve) Computation
 - Flattening Quicksort



1 Implementation of Flat Bulk Operators

- Amdahl's Law
- Work-Depth Asymptotic
- Implementation of Reduce
- Implementation of Scan
- Implementation of Segmented Scan
- Other Second-Order Parallel Operators

2 Nested Data-Parallel Applications

- Sieve: Prime-Numbers Computation
- Nested Parallel Quicksort

3 Flattening Nested Parallelism

- Flattening Recipe (by Map Distribution)
- Re-Writing Rules For Flattening
- Flattening Prime-Number (Sieve) Computation
- Flattening Quicksort



Flattenning

Simple and incomplete recipe for flattening:

- I Normalize the program (think 3-address form);
- II Start distributing outer-maps across its containing let-binding statements;
- III Whenever the body of the map is exactly a map, or a reduce, or a scan, or a iota, or ... apply the corresponding re-write rule.

The intent is to present the gist of flattening, not the full transformation, which is complex and tedious. For example, we do not give the rewrite rules for the cases when the map contains a loop or an if-then-else expression, which themselves contain inner parallelism. Finally, we do not discuss the rules for handling divide-and-conquer recursion.

A more systematic treatment is presented in two DPP lectures:

<https://github.com/diku-dk/dpp-e2025-pub/blob/main/slides/L3and4-irreg-flattening.pdf>



How to Flatten? A Relatively Simple Case

```
let arr = [1, 2, 3, 4] in  
map (\i -> map (+(i+1)) (iota i)) arr  
-- Result: [[2],[3,4],[4,5,6],[5,6,7,8]]
```



How to Flatten? A Relatively Simple Case

```
let arr = [1, 2, 3, 4] in
map (\i -> map (+(i+1)) (iota i)) arr
-- Result: [[2],[3,4],[4,5,6],[5,6,7,8]]
```

I. Normalize the code:

```
map (\i -> let ip1 = i+1 in
            let iot = (iota i) in
            let ip1r= (replicate i ip1) in
            map2 (+) ip1r iot                ) arr
```

II. Distribute the map over every instruction in the body

(bottom-up if $\text{nest} > 2$), where $\mathcal{F}$ denotes the flattening transf, and modify the inputs (results) accordingly.



How to Flatten? A Relatively Simple Case

```
let arr = [1, 2, 3, 4] in
map (\i -> map (+(i+1)) (iota i)) arr
-- Result: [[2],[3,4],[4,5,6],[5,6,7,8]]
```

I. Normalize the code:

```
map (\i -> let ip1 = i+1 in
           let iot = (iota i) in
           let ip1r= (replicate i ip1) in
           map2 (+) ip1r iot                ) arr
```

II. Distribute the map over every instruction in the body

(bottom-up if $\text{nest} > 2$), where $\mathcal{F}$ denotes the flattening transf, and modify the inputs (results) accordingly.

```
 $\mathcal{F}(\text{map } (\lambda i \rightarrow \text{map } (+(i+1)) (iota i)) \text{ arr}) \equiv$ 
1. let ip1s = map (\i -> i+1) arr -- [2, 3, 4, 5]
2. let iots =  $\mathcal{F}(\text{map } (\lambda i \rightarrow (iota i)) \text{ arr})$ 
3. let ip1rs=  $\mathcal{F}(\text{map2 } (\lambda i \text{ ip1} \rightarrow (\text{replicate } i \text{ ip1})) \text{ arr ip1s})$ 
4. in  $\mathcal{F}(\text{map2 } (\lambda \text{ ip1r iot} \rightarrow \text{map2 } (+) \text{ ip1r iot}) \text{ ip1rs iots})$ 
```



How to Flatten? A Relatively Simple Case

According to rule (4) **iota** nested inside a **map**

(assuming `arr = [1,2,3,4]`):

```
2. let iots =  $\mathcal{F}$ (map (\i -> iota i) arr)
```

≡

```
inds = scanexc (+) 0 arr -- [0,1,3,6]
size = (last inds) + (last arr) -- 6 + 4 = 10
flag = scatter (replicate size 0)
      inds arr
--      [1, 2, 0, 3, 0, 0, 4, 0, 0, 0]
tmp  = replicate size 1
iots = sgmScanexc (+) 0 flag tmp --[0, 0, 1, 0, 1, 2, 0, 1, 2, 3]
```



How to Flatten? A Relatively Simple Case

According to rule (3) replicate nested inside a map
(assuming `arr = [1,2,3,4]`):

```
3. let ip1rs=  $\mathcal{F}$ (map2 (\ i ip1 -> replicate i ip1) arr ip1s)
   ≡
vals = scatter (replicate size 0) inds ip1s -- [2, 3, 0, 4, 0, 0, 5, 0, 0, 0]
ip1rs= sgmScaninc (+) 0 flag vals           -- [2, 3, 3, 4, 4, 4, 5, 5, 5, 5]
```

According to rule (2) map nested inside a map

```
 $\mathcal{F}$ (map2 (\ ip1r iot -> map2 (+) ip1r iot) ip1rs iots)
≡
4. result = map (+) ip1rs iots
-- [2, 3, 3, 4, 4, 4, 5, 5, 5, 5]
-- [0, 0, 1, 0, 1, 2, 0, 1, 2, 3]
-- + + + + + + + + + +
-----
-- [2, 3, 4, 4, 5, 6, 5, 6, 7, 8] values
-- [1, 2, 0, 3, 0, 0, 4, 0, 0, 0] flags
```



1 Implementation of Flat Bulk Operators

- Amdahl's Law
- Work-Depth Asymptotic
- Implementation of Reduce
- Implementation of Scan
- Implementation of Segmented Scan
- Other Second-Order Parallel Operators

2 Nested Data-Parallel Applications

- Sieve: Prime-Numbers Computation
- Nested Parallel Quicksort

3 Flattening Nested Parallelism

- Flattening Recipe (by Map Distribution)
- **Re-Writing Rules For Flattening**
- Flattening Prime-Number (Sieve) Computation
- Flattening Quicksort



Nested vs Flattened Parallelism: Scan inside a Map

(1) Scan inside a nested map:

```
map (\row->scaninc (+) 0 row) [[1,3], [2,4,6]]
```

```
≡
```

```
[ scaninc (+) 0 [1,3],    scaninc (+) 0 [2,4,6] ]
```

```
≡
```

```
[ [ 1, 4],                [2, 6, 12] ]
```



Nested vs Flattened Parallelism: Scan inside a Map

(1) Scan inside a nested map:

```
map (\row->scaninc (+) 0 row) [[1,3], [2,4,6]]
≡
[ scaninc (+) 0 [1,3],    scaninc (+) 0 [2,4,6] ]
≡
[ [ 1, 4],                [2, 6, 12] ]
```

becomes a segmented scan, which requires a flag array as arg:
(the shape of the result is the same as the shape of the input array)

```
sgmScaninc (+) 0 [2, 0, 3, 0, 0] [1, 3, 2, 4, 6] ≡ [ 1, 4, 2, 6, 12 ]
```

The flag array `[2, 0, 3, 0, 0]` encodes the fact that the flat-data array `[1, 3, 2, 4, 6]` has two segments:

- one of length 2 starting at index 0
- one of length 3 starting at index 2

(i.e., a non-zero element in the flag array denotes the length of the segment that starts at that point.)



Nested vs Flattened Parallelism: Map inside a Map

(2) Map nested inside a map:

```
map (\row->map f row) [[1,3], [2,4,6]]  
≡  
[ map f [1, 3],      map f [2, 4, 6] ]  
≡  
[ [f(1),f(3)], [f(2),f(4),f(6)] ]
```



Nested vs Flattened Parallelism: Map inside a Map

(2) Map nested inside a map:

```
map (\row->map f row) [[1,3], [2,4,6]]  
≡  
[ map f [1, 3],      map f [2, 4, 6] ]  
≡  
[ [f(1),f(3)], [f(2),f(4),f(6)] ]
```

becomes a map on the flat array:

```
map f [1, 3, 2, 4, 6] ≡ [ f(1), f(3), f(2), f(4), f(6) ]
```

The flag array is assumed known and is preserved [2, 0, 3, 0, 0]



How To Distribute the Segment Size?

Assume flag array: $[2, 0, 3, 0, 0]$.

How do we get $[2, 2, 3, 3, 3]$?



How To Distribute the Segment Size?

Assume flag array: [2, 0, 3, 0, 0].

How do we get [2, 2, 3, 3, 3]?

`sgmScaninc (+) 0 flags flags`



Plan for Flattening Replicate inside a Map

(3) Replicate nested inside a map:

```
map2 (\ n m -> replicate n m) [1,3,2] [7,8,9] ≡  
[ replicate 1 7, replicate 3 8, replicate 2 9 ] ≡  
[ [7], [8,8,8], [9,9] ]
```

How do we flatten `map2 (\ n m -> replicate n m) ns ms` ?

0 What is the shape of the result?



Plan for Flattening Replicate inside a Map

(3) Replicate nested inside a map:

```
map2 (\ n m -> replicate n m) [1,3,2] [7,8,9] ≡  
[ replicate 1 7, replicate 3 8, replicate 2 9 ] ≡  
[ [7], [8,8,8], [9,9] ]
```

How do we flatten `map2 (\ n m -> replicate n m) ns ms` ?

- 0 What is the shape of the result? The shape is `ns`
- 1 Create the flag array of the result array:
[1, 1, 0, 0, 1, 0]



Plan for Flattening Replicate inside a Map

(3) Replicate nested inside a map:

```
map2 (\ n m -> replicate n m) [1,3,2] [7,8,9] ≡  
[ replicate 1 7, replicate 3 8, replicate 2 9 ] ≡  
[ [7], [8,8,8], [9,9] ]
```

How do we flatten `map2 (\ n m -> replicate n m) ns ms` ?

- 0 What is the shape of the result? The shape is `ns`
- 1 Create the flag array of the result array:
[1, 1, 0, 0, 1, 0]
- 2 Scatter each element of `ms` the beginning of each segment:
[7, 8, 0, 0, 9, 0]



Plan for Flattening Replicate inside a Map

(3) Replicate nested inside a map:

```
map2 (\ n m -> replicate n m) [1,3,2] [7,8,9] ≡
[ replicate 1 7, replicate 3 8, replicate 2 9 ] ≡
[ [7], [8,8,8], [9,9] ]
```

How do we flatten map2 (\ n m -> replicate n m) ns ms ?

- 0 What is the shape of the result? The shape is ns
- 1 Create the flag array of the result array:
[1, 1, 0, 0, 1, 0]
- 2 Scatter each element of ms the beginning of each segment:
[7, 8, 0, 0, 9, 0]
- 3 Use a segmented scan to move the element along its segment
[7, 8, 8, 8, 9, 9]



Nested vs Flattened Parallelism: Replicate in a Map

(3) Replicate nested inside a map:

```
map2 (\ n m -> replicate n m) [1,3,2] [7,8,9] ≡  
[ replicate 1 7, replicate 3 8, replicate 2 9 ] ≡  
[ [7], [8,8,8], [9,9] ]
```



Nested vs Flattened Parallelism: Replicate in a Map

(3) Replicate nested inside a map:

```
map2 (\ n m -> replicate n m) [1,3,2] [7,8,9] ≡
[ replicate 1 7, replicate 3 8, replicate 2 9 ] ≡
[ [7], [8,8,8], [9,9] ]
```

becomes a composition of scans and scatter:

```
-- ns = [1,3,2], ms = [7,8,9]
1. inds = scanexc (+) 0 ns           -- [0,1,4]
2. size = (last inds) + (last ns)   -- 4 + 2 = 6
3. flag = scatter (replicate size 0) inds ns  -- [1, 3, 0, 0, 2, 0]
4. vals = scatter (replicate size 0) inds ms  -- [7, 8, 0, 0, 9, 0]
5. sgmScaninc (+) 0 flag vals        -- [7, 8, 8, 8, 9, 9]
```

- 0. the shape of the result is ns
- 1. builds the indices at which segments start
- 2. get the size of the flat array (equivalent to summing ns)
- 3-4. write the array elems at the position where a segment starts
- 5. distribute the start-elem of a segment throughout the segment.

- **Implementation shortcomings:**



Nested vs Flattened Parallelism: Replicate in a Map

(3) Replicate nested inside a map:

```
map2 (\ n m -> replicate n m) [1,3,2] [7,8,9] ≡
[ replicate 1 7, replicate 3 8, replicate 2 9 ] ≡
[ [7], [8,8,8], [9,9] ]
```

becomes a composition of scans and scatter:

```
-- ns = [1,3,2], ms = [7,8,9]
1. inds = scanexc (+) 0 ns           -- [0,1,4]
2. size = (last inds) + (last ns)   -- 4 + 2 = 6
3. flag = scatter (replicate size 0) inds ns  -- [1, 3, 0, 0, 2, 0]
4. vals = scatter (replicate size 0) inds ms  -- [7, 8, 0, 0, 9, 0]
5. sgmScaninc (+) 0 flag vals        -- [7, 8, 8, 8, 9, 9]
```

- 0. the shape of the result is ns
- 1. builds the indices at which segments start
- 2. get the size of the flat array (equivalent to summing ns)
- 3-4. write the array elems at the position where a segment starts
- 5. distribute the start-elem of a segment throughout the segment.
 - **Implementation shortcomings:** replicate 0 7?



Nested vs Flattened Parallelism: Replicate in a Map

(3) Replicate nested inside a map:

```
map2 (\ n m -> replicate n m) [1,3,2] [7,8,9] ≡
[ replicate 1 7, replicate 3 8, replicate 2 9 ] ≡
[ [7], [8,8,8], [9,9] ]
```

becomes a composition of scans and scatter:

```
-- ns = [1,3,2], ms = [7,8,9]
1. inds = scanexc (+) 0 ns           -- [0,1,4]
2. size = (last inds) + (last ns)    -- 4 + 2 = 6
3. flag = scatter (replicate size 0) inds ns  -- [1, 3, 0, 0, 2, 0]
4. vals = scatter (replicate size 0) inds ms  -- [7, 8, 0, 0, 9, 0]
5. sgmScaninc (+) 0 flag vals        -- [7, 8, 8, 8, 9, 9]
```

- 0. the shape of the result is ns
- 1. builds the indices at which segments start
- 2. get the size of the flat array (equivalent to summing ns)
- 3-4. write the array elems at the position where a segment starts
- 5. distribute the start-elem of a segment throughout the segment.
 - **Implementation shortcomings:** replicate 0 ? $\text{sgmScan}^{\text{inc}} (+)$?
 - see mkFlagArray function in lecture notes.



Plan for Flattening Replicate inside a Map

(4) Iota nested inside a map $((\text{iota } n) \equiv [0, \dots, n-1])$:

```
map (\i -> iota i) [1,3,2]  $\equiv$ 
[ iota 1, iota 3, iota 2 ]  $\equiv$  [ [0], [0,1,2], [0,1] ]
```

How do we flatten map $(\lambda n \rightarrow \text{iota } n)$ ns ?

0 What is the shape of the result?



Plan for Flattening Replicate inside a Map

(4) Iota nested inside a map $((\text{iota } n) \equiv [0, \dots, n-1])$:

```
map (\i -> iota i) [1,3,2]  $\equiv$ 
[ iota 1, iota 3, iota 2 ]  $\equiv$  [ [0], [0,1,2], [0,1] ]
```

How do we flatten map $(\lambda n \rightarrow \text{iota } n)$ ns ?

0 What is the shape of the result? The shape is ns

1 Create the flag array of the result array:

[1, 1, 0, 0, 1, 0]



Plan for Flattening Replicate inside a Map

(4) `iota` nested inside a `map` ($(\text{iota } n) \equiv [0, \dots, n-1]$):

```
map (\i -> iota i) [1,3,2]  $\equiv$ 
[ iota 1, iota 3, iota 2 ]  $\equiv$  [ [0], [0,1,2], [0,1] ]
```

How do we flatten `map (\ n -> iota n) ns` ?

- 0 What is the shape of the result? The shape is `ns`
- 1 Create the flag array of the result array:
[1, 1, 0, 0, 1, 0]
- 2 Create an array of ones of matching size with the result:
[1, 1, 1, 1, 1, 1]



Plan for Flattening Replicate inside a Map

(4) `iota` nested inside a map $((iota\ n) \equiv [0, \dots, n-1])$:

`map (\i -> iota i) [1,3,2]  $\equiv$`

`[iota 1, iota 3, iota 2]  $\equiv$  [ [0], [0,1,2], [0,1] ]`

How do we flatten `map (\ n -> iota n) ns` ?

- 0 What is the shape of the result? The shape is `ns`
- 1 Create the flag array of the result array:
[1, 1, 0, 0, 1, 0]
- 2 Create an array of ones of matching size with the result:
[1, 1, 1, 1, 1, 1]
- 3 Use a segmented exclusive scan: [0, 0, 1, 2, 0, 1]



Nested vs Flattened Parallelism: Iota in a Map

(4) Iota nested inside a map $((\text{iota } n) \equiv [0, \dots, n-1])$:

```
map (\i -> iota i) [1,3,2]  $\equiv$   
[iota 1, iota 3, iota 2]  $\equiv$  [ [0], [0,1,2], [0,1] ]
```



Nested vs Flattened Parallelism: Iota in a Map

(4) Iota nested inside a map $((iota\ n) \equiv [0, \dots, n-1])$:

```
map (\i -> iota i) [1,3,2]  $\equiv$ 
[ iota 1, iota 3, iota 2 ]  $\equiv$  [ [0], [0,1,2], [0,1] ]
```

becomes a composition of scans and scatter:

Note that $iota\ n \equiv scan^{exc}\ (+)\ 0\ (replicate\ n\ 1)$

```
1. arr  = [1, 3, 2]
2. inds = scanexc (+) 0 arr      -- [0,1,4]
3. size = (last inds) + (last arr) -- 4 + 2 = 6
4. flag = scatter (replicate size 0)
           inds -- [0,1,4]
           arr  -- [1,3,2]
--
[1, 3, 0, 0, 2, 0]
5. tmp  = replicate size 1 --[1, 1, 1, 1, 1, 1]
6. sgmScanexc (+) 0 flag tmp --[0, 0, 1, 2, 0, 1]
```

2. builds the indices at which segment start
3. get the size of the flat array (equivalent to summing arr)
4. write the array elems at the position where a segment starts
6. segmented scan an array of ones.



Plan for Flattening Reduce inside a Map

(5) Reduce Inside a Map or Segmented Reduce:

```
let arr = [[1, 3, 4], [6, 7]] in  
map (\x -> reduce (+) 0 x) arr  
-- should result in [8, 13]
```

How do we flatten `map (\x -> reduce op e x) arr` ?

- 0 We know the shape of the input `[3, 2]`.
The result array has length equal to



Plan for Flattening Reduce inside a Map

(5) Reduce Inside a Map or Segmented Reduce:

```
let arr = [[1, 3, 4], [6, 7]] in  
map (\x -> reduce (+) 0 x) arr  
-- should result in [8, 13]
```

How do we flatten `map (\ x -> reduce op e x) arr` ?

0 We know the shape of the input `[3, 2]`.

The result array has length equal to the number of rows of `arr`.

1 Create the flag array of the input array:

`[1, 0, 0, 1, 0]`



Plan for Flattening Reduce inside a Map

(5) Reduce Inside a Map or Segmented Reduce:

```
let arr = [[1, 3, 4], [6, 7]] in  
map (\x -> reduce (+) 0 x) arr  
-- should result in [8, 13]
```

How do we flatten $\text{map } (\lambda x \rightarrow \text{reduce op e } x) \text{ arr}$?

0 We know the shape of the input [3, 2].

The result array has length equal to the number of rows of arr.

1 Create the flag array of the input array:

[1, 0, 0, 1, 0]

2 Perform an inclusive segmented scan:

[1, 4, 8, 6, 13]

3 The result array consist of the last element of each segment.

How do we compute the corresponding indices?



Plan for Flattening Reduce inside a Map

(5) Reduce Inside a Map or Segmented Reduce:

```
let arr = [[1, 3, 4], [6, 7]] in
map (\x -> reduce (+) 0 x) arr
-- should result in [8, 13]
```

How do we flatten `map (\ x -> reduce op e x) arr` ?

- 0 We know the shape of the input `[3, 2]`.
The result array has length equal to the number of rows of `arr`.
- 1 Create the flag array of the input array:
`[1, 0, 0, 1, 0]`
- 2 Perform an inclusive segmented scan:
`[1, 4, 8, 6, 13]`
- 3 The result array consist of the last element of each segment.
How do we compute the corresponding indices?
By an inclusive scan on the shape `[3,2]` minus one:
`[3-1, 5-1] = [2, 4]`
- 4 How do we extract the last element of each segment?



Plan for Flattening Reduce inside a Map

(5) Reduce Inside a Map or Segmented Reduce:

```
let arr = [[1, 3, 4], [6, 7]] in
map (\x -> reduce (+) 0 x) arr
-- should result in [8, 13]
```

How do we flatten `map (\ x -> reduce op e x) arr` ?

- 0 We know the shape of the input `[3, 2]`.
The result array has length equal to the number of rows of `arr`.
- 1 Create the flag array of the input array:
`[1, 0, 0, 1, 0]`
- 2 Perform an inclusive segmented scan:
`[1, 4, 8, 6, 13]`
- 3 The result array consist of the last element of each segment.
How do we compute the corresponding indices?
By an inclusive scan on the shape `[3,2]` minus one:
`[3-1, 5-1] = [2, 4]`
- 4 How do we extract the last element of each segment?
By a gather (map) operation applied to the previous array.



Nested vs Flattened Parallelism: Reduce Inside Map

(5) Reduce Inside a Map or Segmented Reduce:

```
let arr = [[1, 3, 4], [6, 7]] in  
map (\x -> reduce (+) 0 x) arr  
-- should result in [8, 13]
```



Nested vs Flattened Parallelism: Reduce Inside Map

(5) Reduce Inside a Map or Segmented Reduce:

```
let arr = [[1, 3, 4], [6, 7]] in
map (\x -> reduce (+) 0 x) arr
-- should result in [8, 13]
```

translates to a **scan-gather** composition:

```
1. shp      = [3, 2]
2. flags    = [1, 0, 0, 1, 0]
3. arr      = [1, 3, 4, 6, 7]
4. n        = length arr
5. indsp1   = scaninc (+) 0 shp          -- [3, 5]
6. sc_arr   = sgmScaninc (+) 0 flags arr -- [1, 4, 8, 6, 13]
7. res      = map (\ ip1 -> sc_arr[ip1-1]) indsp1
```

How do we handle empty segments?



Nested vs Flattened Parallelism: Reduce Inside Map

(5) Reduce Inside a Map or Segmented Reduce:

```
let arr = [[1, 3, 4], [6, 7]] in
map (\x -> reduce (+) 0 x) arr
-- should result in [8, 13]
```

translates to a **scan-gather** composition:

```
1. shp      = [3, 2]
2. flags    = [1, 0, 0, 1, 0]
3. arr      = [1, 3, 4, 6, 7]
4. n        = length arr
5. indsp1   = scaninc (+) 0 shp          -- [3, 5]
6. sc_arr   = sgmScaninc (+) 0 flags arr -- [1, 4, 8, 6, 13]
7. res      = map (\ ip1 -> sc_arr[ip1-1]) indsp1
```

How do we handle empty segments?

We can pass the shape array as argument to the map as well, and return the neutral element if the segment length is 0.



1 Implementation of Flat Bulk Operators

- Amdahl's Law
- Work-Depth Asymptotic
- Implementation of Reduce
- Implementation of Scan
- Implementation of Segmented Scan
- Other Second-Order Parallel Operators

2 Nested Data-Parallel Applications

- Sieve: Prime-Numbers Computation
- Nested Parallel Quicksort

3 Flattening Nested Parallelism

- Flattening Recipe (by Map Distribution)
- Re-Writing Rules For Flattening
- Flattening Prime-Number (Sieve) Computation
- Flattening Quicksort



How Does One Flatten Prime Numbers?

The important bit with nested parallelism:

```
sqrt_primes = primesOpt (sqrt (fromIntegral n))
nested = map (\p -> let m = (n `div` p)
                  in map (\j -> j*p) [2..m]
                ) sqrt_primes
not_primes = reduce (++) [] nested
```



How Does One Flatten Prime Numbers?

The important bit with nested parallelism:

```
sqrt_primes = primesOpt (sqrt (fromIntegral n))
nested = map (\p -> let m = (n `div` p)
                    in map (\j -> j*p) [2..m]
                    ) sqrt_primes
not_primes = reduce (++) [] nested
```

Normalize the nested map:

```
sqrt_primes = primesOpt (sqrt (fromIntegral n))
nested = map (\p ->
    let m      = n `div` p      in      -- distribute map
    let mm1    = m - 1          in      -- distribute map
    let iot    = iota mm1       in      --  $\mathcal{F}$  rule 4
    let twom   = map (+2) iot    in      --  $\mathcal{F}$  rule 2
    let rp     = replicate mm1 p in      --  $\mathcal{F}$  rule 3
    in map (\(j,p) -> j*p) (zip twom rp) --  $\mathcal{F}$  rule 2
    ) sqrt_primes
not_primes = reduce (++) [] nested      -- ignore, already flat
```

Flattening PrimeOpt is part of Weekly Assignment 2!



1 Implementation of Flat Bulk Operators

- Amdahl's Law
- Work-Depth Asymptotic
- Implementation of Reduce
- Implementation of Scan
- Implementation of Segmented Scan
- Other Second-Order Parallel Operators

2 Nested Data-Parallel Applications

- Sieve: Prime-Numbers Computation
- Nested Parallel Quicksort

3 Flattening Nested Parallelism

- Flattening Recipe (by Map Distribution)
- Re-Writing Rules For Flattening
- Flattening Prime-Number (Sieve) Computation
- Flattening Quicksort



Recounting Quicksort

Recount the classic nested-parallel definition:

```
nestedQuicksort :: [a] -> [a]
nestedQuicksort arr =
  if (length arr) <= 1 then arr else
  let i = getRand (0, (length arr) - 1)
      a = arr !! i
      s1 = filter (\x -> (x < a)) arr
      s2 = filter (\x -> (x >= a)) arr
  in  (nestedQuicksort s1) ++ (nestedQuicksort s2)
-- can be re-written as:
-- rs = map nestedQuicksort [s1, s2]
-- in (rs !! 0) ++ (rs !! 1)
```



Normalizing Quicksort

Key Idea: write a function with the semantics of

`map nestedQuicksort`, i.e., it operates on array of arrays, and, for simplicity, use `partition2 :: ( $\alpha \rightarrow \text{Bool}$ )  $\rightarrow [\alpha] \rightarrow ([\alpha], [\text{Int}])$` .

```
quicksortlift :: [[a]] -> [[a]]
quicksortlift arrofarrs =
  map (\arr ->
    if (length arr) < 2 then arr else
    let i = getRand (0, (length arr) - 1)
        a = arr !! i
        (s, flag) = partition2 (<a) arr
        (s1, s2) = split flag[0] s
        rs = map nestedQuicksort [s1, s2] --  $\equiv$  quicksortlift [s1, s2]
    in concat rs
  ) arrofarrs
```

`(length arr) < 2` will not work correctly if the input array has duplicated elements (?)

What should the result be of distributing the `map` over `concat (map nestedQuicksort [s1,s2])`?



Normalizing Quicksort

- Try to distribute the outer `map` across the inner code;
- The recursion can be re-written as a loop in which the stopping condition is that the whole array is sorted; this can be expressed as a map-reduce composition;
- Distributing the outer `map` over `concat (map nestedQuicksort [s1,s2])` results in `quicksortlift` being applied to a differently shaped array, i.e., twice as many segments as before.
- The difficult step is to flatten `map (partition2)`.

